

Data Mining

ASSOCIATION RULES

BY: IVETA MRÁZOVÁ

METHODS: ASSOCIATION RULES

Contents:

- Market Basket Analysis
- Minimum Item Support
- Class Association Rules
- Sequential Pattern Mining
- Frequent Pattern Growth

Contents:

- Market Basket Analysis (MBA)
 - Association rules
 - Support, confidence, and lift
 - Algorithm APRIORI
- Minimum Item Support (MIS)
 - The MS-Apriori algorithm
- Class Association Rules (CAR)
- Sequential Pattern Mining
 - Basic concepts and an example
 - Generalized sequential pattern (GSP) mining
- Frequent Pattern Growth (FP-Growth)
 - FP-Tree construction
 - Frequent Itemset Generation
 - Prefix (path) sub-tree
 - Conditional FP-tree

METHODS: ASSOCIATION RULES

Contents:

- **Market Basket Analysis**
- *Minimum Item Support*
- *Class Association Rules*
- *Sequential Pattern Mining*
- *Frequent Pattern Growth*

Contents:

- **Market Basket Analysis (MBA)**
 - **Association rules**
 - **Support, confidence, and lift**
 - **Algorithm APRIORI**
- *Minimum Item Support (MIS)*
 - *The MS-Apriori algorithm*
- *Class Association Rules (CAR)*
- *Sequential Pattern Mining*
 - *Basic concepts and an example*
 - *Generalized sequential pattern (GSP) mining*
- *Frequent Pattern Growth (FP-Growth)*
 - *FP-Tree construction*
 - *Frequent Itemset Generation*
 - *Prefix (path) sub-tree*
 - *Conditional FP-tree*



Market Basket Analysis (MBA)

Sales analysis:

What items occur together in the 'basket'?

Results:

- expressed in the form of rules
- can be immediately applied

Application:

- Planning and layout of shops
- offer of coupons, limited discounts
- 'packaging' of products





Association Rules

How are the respective products related?

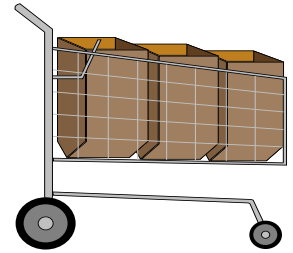
Association rules should be:

- **easy-to-understand:** after there is a relationship found, it can be easily verified
- **applicable:** they comprise useful information, that can lead to further interventions

Association rules should be NOT:

- **trivial:** everyone already knows the results
- **inexplicable:** there does not exist an explanation for them, and they do not lead to an action

MBA for shop comparison



Virtual items:

- Indicate the type of the shop, where the transaction took place
- Do not correspond to any product or service

A comparison of new and already existing shops:

1. Provide transaction data from new shops for a certain time
2. Provide about the same amount of data from existing shops
3. Use MBA to find association rules in both sets of transactions
4. In particular, consider association rules with virtual items

MBA – How to do that?



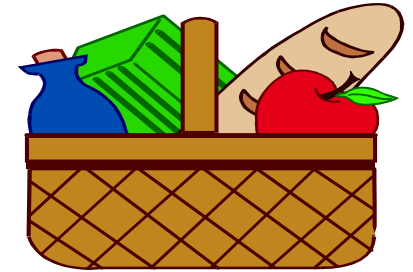
Item - product or service offer

Transaction contains one or more *items*

Frequency table

- indicates the number of co-occurrences of any two *items* within the performed *transactions* (i.e., how many times were these two products purchased together)
- the values on its diagonal correspond to *the number of transactions* that contain the respective item

MBA - example



Transactions in a grocery shop:

Customer	Items
1	bread, butter
2	milk, bread, butter
3	bread, coffee
4	bread, butter, coffee
5	coffee, butter

Frequency of sold products:

	bread	butter	milk	coffee
bread	4	3	1	2
butter	3	4	1	2
milk	1	1	1	0
coffee	2	2	0	3

The character of sales becomes apparet from the frequency table:

- Bread and butter tend to be bought together.
- Milk is never purchased together with coffee.

MBA – association rules



The form of the rules:

IF Condition THEN Conclusion

(~ Rule_r : IF Item_i THEN Item_j .)

Questions:

- How good are the found association rules?
 - support
 - confidence
 - lift
- How to find the association rules automatically?



Support and confidence

Support: *How often can we use the rule?*

$$\text{Support}(\text{Rule}_r) = \frac{\text{Nr_of_transactions_that_contain_i_and_j}}{\text{Nr_of_all_transactions}} \bullet 100 \%$$

Confidence: *How reliable are the results of the rules?*

$$\text{Confidence}(\text{Rule}_r) = \frac{\text{Nr_of_transactions_that_contain_i_and_j}}{\text{Nr_of_transactions_that_contain_i}} \bullet 100 \%$$



Support and confidence - example

Rule 1: *If the customer buys bread
then he buys also butter.*

Rule 2: *If the customer buys coffee
then he buys also butter.*

$$\text{Support (Rule_1)} = 3 / 5 \cdot 100 \% = 60 \%$$

$$\text{Support (Rule_2)} = 2 / 5 \cdot 100 \% = 40 \%$$

$$\text{Confidence (Rule_1)} = 3 / 4 \cdot 100 \% = 75 \%$$

$$\text{Confidence (Rule_2)} = 2 / 3 \cdot 100 \% = 66 \%$$

The lift of a rule



Lift: *How many times is it better to use the rule for prediction instead of just assuming its result?*

$$\text{Lift}(\text{Rule}_r) = \frac{p(i_a_j)}{p(i) \cdot p(j)}$$

If it holds that $\text{Lift} < 1$:

- ♦ the rule is worse than a random choice for prediction
- ♦ **NEGATION** of the result can yield a better rule

IF *Condition* THEN NOT *Conclusion*.



The lift of a rule – an example

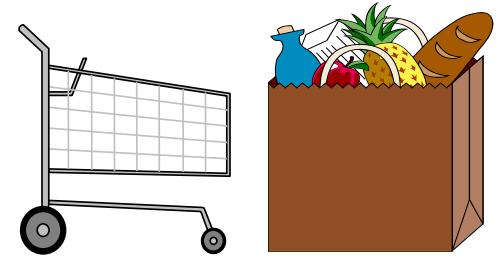
Rule: *If the customer buys milk then he buys also butter.*

$$\text{Support (Rule)} = 1 / 5 \cdot 100 \% = 20 \%$$

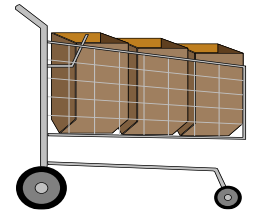
$$\text{Confidence (Rule)} = 1 / 1 \cdot 100 \% = 100 \%$$

$$\text{Lift (Rule)} = (1 / 5) / ((1 / 5) \cdot (4 / 5)) = 5 / 4 = 1.25$$

The main steps of MBA



- **Choose** appropriate **items** at an adequate level
- **Generate rules** based on the frequency table
 - compute (conditional) probabilities of item occurrence and occurrence of their combinations in the transactions
 - Restrict the search by a threshold value put on support
- **Find the best rules** through analyzing the probabilities
 - Overcome the limits given by the number of items and their combinations in 'interesting' transactions



MBA – the choice of appropriate items

Obtaining the transaction data:

- Often a worse quality, requires a more extensive pre-processing
- The relevance of the items can change with time
- An adequate level of processing:
 - A growing number of combinations of the respective items
 - The results can be immediately applied (specific items)
 - Rules with a sufficiently high support (frequent occurrence in the data set)

Taxonomy: the hierarchy of the categories



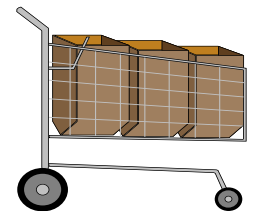
MBA – the complexity of generated rules:

- In the beginning, use more general items
- Later, generate rules for specific items based only on the transactions that contain these items

MBA – Applicable results:

The considered items should occur in roughly the same number of transactions:

- Move rare items to higher levels in the taxonomy (where they appear relatively more frequently)
- Leave the more common items at lower levels (in order to not allow the most frequent items to dominate the rules)



Virtual items:

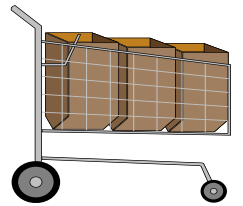
exceed the framework of a traditional taxonomy

- Blur the boundary between certain types of products in original items
 - E.g., (company) brands - Calvin Klein
- Can contain information about the transaction itself
 - **anonymous** (day of week, time, etc.)
 - **signed** (information about the customers and their behavior)
- Can impact redundancies in the rules
 - Just one virtual item corresponds to the items from the taxonomy (*'If product_Škoda then Škoda.'*)
 - Both the virtual and general items occur in the rule at the same time (*'If product_Škoda and small car then tent'* instead of *'If Fabia then tent'*)



MBA – generating of the rules

- **Compute the frequency table:**
 - Provides the information about, what combinations of the respective items appear most frequently in the transactions
 - Can be used to determine the basic probabilities necessary to assess the impact of the generated rules
- **Obtain useful rules:**
 - The lift should be bigger than 1
 - A low lift can be increased by negating the rules
 - Negated rules might be less applicable than the original ones
 - reduce the number of generated rules - **PRUNING**



Pruning according to minimum support

Elimination of less frequent items

- The undertaken actions should concern *enough transactions*
- Two options:
 - Elimination of rare items (and a subsequent elimination of corresponding rules)
 - Use a taxonomy to create general items (generalized items should meet the set criteria - threshold)
- *variable minimum support* => a cascade effect

MBA – Algorithm APRIORI (R. Agrawal)

- Generation of the association rules
 - Looking for frequent item-sets

~ Frequent item-sets:

- Combinations (~ conjunctions) of categories, that reach a pre-set frequency on the data ~ **'minsup' support**
- When looking for combinations of the length **k** , we use the known combinations of the length **$k - 1$** ~ **breadth-first generation of combinations**
- To generate the combinations of the length **k** , we demand all their sub-combinations of the length **$k - 1$** to fulfill the preset frequency requirement

MBA – Algorithm APRIORI (continued)

~ **Frequent item-sets** (continued):

- After finding the combinations with a satisfiable frequency, the association rules will be created
 - (*supercombination_frequency* $\leq$ *combination_frequency*)

→ Each combination **Comb** will be divided into all possible pairs of sub-combinations **Ant** and **Suc** such that:

$$\mathbf{Suc} = \mathbf{Comb} - \mathbf{Ant}$$

$$(\mathbf{Ant} \cap \mathbf{Suc} = \{ \} \text{ and } \mathbf{Ant} \cup \mathbf{Suc} = \mathbf{Comb})$$

→ Then, for the considered rule **Ant** $\Rightarrow$ **Suc** :

- **support** corresponds to the frequency of the combination **Comb**
- **confidence** corresponds to the ratio of the frequencies of the combinations **Comb** and **Ant**

MBA – Algorithm APRIORI

Step 1: Assign to L_1 all the categories that reach at least the required frequency

Step 2: Set $k = 2$

Step 3: Until $L_{k-1} \neq \{\}$

Step 3.1: Based on L_{k-1} and by means of the function **APRIORI-GEN**, generate a set of candidates C_k

Step 3.2: Assign to L_k all those combinations from C_k , that reached at least the required frequency

Step 3.3: Increment k

The function APRIORI-GEN (L_{k-1})

Step 1: For all pairs of combinations

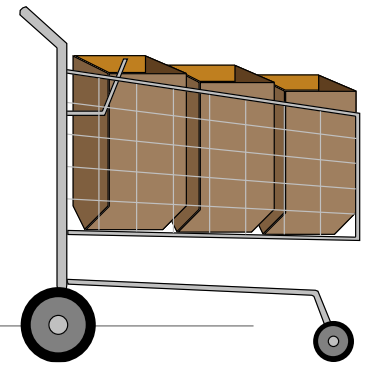
Comb_p, **Comb_q** from L_{k-1}

Step 1.1: If **Comb_p** and **Comb_q** match in $k - 2$ categories, add **Comb_p** $\wedge$ **Comb_q** to C_k

Step 2: For each combination **Comb** from C_k

Step 2.1: If any of their sub-combinations of the length $k - 1$ is not contained in L_{k-1} , remove **Comb** from C_k

MBA – Dissociation rules

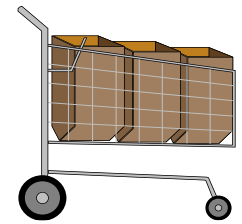


Rule: IF A AND NOT B THEN C .

- Introduce new items complementary to the original items
- If the transaction does not contain the original item, it will contain its negation

■ **Disadvantages:**

- Twice as many items
- Increased size of transactions
- Negated items occur more often than the original ones (and the rules with all the items negated are more difficult to exploit: 'IF NOT A AND NOT B THEN NOT C .')



Time-series analysis by means of MBA

■ A cause – effect analysis:

- Information about the time, or event sequence to determine when the transactions occurred (one in relation to the other)
- an identification of the customer may be necessary

■ A transfer of the problem to MBA:

- Introduction of new items to transactions that occurred before the considered event (for the *causes*) and after the considered event (for the *effects*); duplicate items are subsequently removed from the transactions
- **A window:** 'snapshot' of all the data, that occurred during a certain time period (e.g., all transactions from the last month)
 - Trends for rare items

Advantages of MBA



- Provides clear and understandable results
 - ***IF - THEN - rules that are immediately applicable***
- ***Data mining*** (without target output values)
 - Important when processing large amounts of data without additional a-priori knowledge
- Allows to process ***data of a variable length***
- ***Simple and easy-to-understand*** computation
 - X Computational **costs grow exponentially** with the number of items!

Drawbacks of MBA



- Computational costs:

- the space of association rules is **exponential, $O(2^m)$** ,
 - m is the number of the considered items
- Taxonomies and virtual items are necessary

- Difficulties to determine an adequate number of items:

- the items should have about the same frequency
- rare items shall be considered on higher levels in the taxonomy and shall involve variable threshold values for minimum support pruning
 - => high minimum support and confidence can be used to prune less useful and too general items

X ... still, MBA tends to produce excessive numbers of rules!

– thousands, tens of thousands, millions, ...

METHODS: ASSOCIATION RULES

Contents:

- *Market Basket Analysis*
- **Minimum Item Support**
- *Class Association Rules*
- *Sequential Pattern Mining*
- *Frequent Pattern Growth*

Contents:

- *Market Basket Analysis (MBA)*
 - *Association rules*
 - *Support, confidence, and lift*
 - *Algorithm APRIORI*
- **Minimum Item Support (MIS)**
 - **The MS-Apriori algorithm**
- *Class Association Rules (CAR)*
- *Sequential Pattern Mining*
 - *Basic concepts and an example*
 - *Generalized sequential pattern (GSP) mining*
- *Frequent Pattern Growth (FP-Growth)*
 - *FP-Tree construction*
 - *Frequent Itemset Generation*
 - *Prefix (path) sub-tree*
 - *Conditional FP-tree*

MBA – Problems with association mining

- **Single minsup:** assumes that all items in the data are of the same nature and/or have similar frequencies.
- X Yet in many applications, some items occur quite frequently in the data, while others appear rather rarely.
 - e.g., in a supermarket, people buy *food processor* and *cooking pan* much less frequently than they buy *bread* and *milk*.

MBA – Rare Item Problem

- If the frequencies of items vary a lot, we might encounter **two problems**:
 - If **minsup** is set **too high**, those rules that involve rare items will not be found.
 - To find rules that involve both frequent and rare items, **minsup** should be set **very low**.
 - => **This may cause combinatorial explosion** because those frequent items will be associated with one another in all possible ways.

Multiple minsups model

- The minimum support of a rule is expressed in terms of **minimum item supports (MIS)** of the items that appear in the rule.
- Each item can have a minimum item support.
- By providing different MIS values for different items, the user can express different support requirements for different rules.
- To prevent very frequent items and very rare items from appearing in the same itemsets, we introduce a **support difference constraint**:

$$\max_{i \in S} \{sup(i)\} - \min_{i \in S} \{sup(i)\} \leq \varphi,$$

where $0 \leq \varphi \leq 1$ is the user-specified **maximum support difference**; it is the same for all itemsets.

Minsup of a rule

- Let $MIS(i)$ be the minimum item support (MIS) value of item i . The *minsup* of a rule R is the lowest MIS value of the items in the rule.
- i.e., a rule $R: a_1, a_2, \dots, a_k \rightarrow a_{k+1}, \dots, a_r$ satisfies its minimum support if its actual support is $\geq \min(MIS(a_1), MIS(a_2), \dots, MIS(a_r))$.

Minsup of a rule – an example

- Consider the following items:

bread, shoes, clothes

- The user-specified MIS values are as follows:

$MIS(bread) = 2\%$ $MIS(shoes) = 0.1\%$ $MIS(clothes) = 0.2\%$

- The following **rule does not satisfy** its *minsup*:

R1: clothes → bread [sup=0.15%,conf =70%]

sup (R1) = 0.15% < MIS(clothes) = 0.2%

- The following **rule satisfies** its *minsup*:

R2: clothes → shoes [sup=0.15%,conf =70%]

sup (R2) = 0.15% ≥ MIS(shoes) = 0.1%

Downward closure property

- **Downward closure property** ($\sim$ the key idea for a-priori):
any subsets of a frequent itemset are also frequent itemsets

X In the new model, **this property no longer holds**

- Minimum item supports, namely, allow having higher minimum supports for rules that involve only frequent items, and having lower minimum supports for rules with less frequent items.
- e.g., consider four items – 1, 2, 3 and 4 in a database. Their minimum item supports are

MIS(1) = 10% MIS(2) = 20%

MIS(3) = 5% MIS(4) = 6%

{1, 2} with support 9% is infrequent, but {1, 2, 3} and {1, 2, 4} could be frequent => **we cannot discard {1, 2}**

How to deal with the problem?

- Sort all items in I according to their MIS values in an ascending order; I denotes the set of all items
- The order is then fixed and used for all subsequent operations throughout the algorithm in each itemset.
- Each itemset w is of the following form:

$$\{ w[1], w[2], \dots, w[k] \},$$

consisting of the items, $w[1], w[2], \dots, w[k]$,

where $MIS(w[1]) \leq MIS(w[2]) \leq \dots \leq MIS(w[k])$.

The MS-Apriori algorithm

(MS stands for Multiple Minimum Supports)

```

Algorithm MSapriori( $T, MS, \varphi$ )           //  $MS$  stores all MIS values,  $\varphi$  is for support difference constraint
 $M \leftarrow \text{sort}(I, MS);$                  // sorting of the items according to  $MIS(i)'$  s stored in  $MS$ 
 $L \leftarrow \text{init-pass}(M, T);$            // make the first pass over the data set  $T$ ;  $L$  is the set of seeds
 $F_1 \leftarrow \{\{i\} \mid i \in L, i.\text{count}/n \geq MIS(i)\};$  //  $n$  is the size of  $T$ , i. e., the number of transactions in  $T$ 
for ( $k = 2; F_{k-1} \neq \emptyset; k++$ ) do    //  $F_{k-1}$  is the set of frequent  $k$ -itemsets
    if  $k=2$  then                          // a special case, for  $k=2$ ,  $L$  is necessary,  $L$  is the set of seeds
         $C_k \leftarrow \text{level2-candidate-gen}(L, \varphi)$  // generates the set of candidates (with  $L$ )
    else  $C_k \leftarrow \text{MSCandidate-gen}(F_{k-1}, \varphi);$  // generates the set of candidates (with  $F_{k-1}$ )
    end;
    for each transaction  $t \in T$  do          // update of support counts
        for each candidate  $c \in C_k$  do
            if  $c$  is contained in  $t$  then    //  $c$  is a subset of  $t$ 
                 $c.\text{count}++;$ 
                if  $c - \{c[1]\}$  is contained in  $t$  then //  $c$  without the first term
                     $c.\text{tailCount}++$         // used in rule generation; if rule generation is not
                                                // required, this part of code can be deleted
            end
        end
         $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq MIS(c[1])\}$  // identification of frequent itemsets ( $F_k$ )
    end
return  $F \leftarrow \bigcup_k F_k;$            //  $F$  is the set of frequent itemsets

```

Candidate itemset generation

- Special treatment steps needed:

- Sorting the items according to their MIS values: $M \leftarrow \text{sort}(I, MS);$
- First pass over data: $L \leftarrow \text{init-pass}(M, T);$
- Candidate generation at level 2:
 - if** $k = 2$ **then** $C_k \leftarrow \text{level2-candidate-gen}(L, \varphi)$
- Pruning step in level k ($k > 2$) candidate generation:
 - else** $C_k \leftarrow \text{MSCandidate-gen}(F_{k-1}, \varphi);$
- Then, the algorithm scans the data and updates support counts of the candidates in C_k and their candidates without the first item (to be used in rule generation):
 - if** $c - \{c[1]\}$ is contained in t **then** $c.\text{tailCount}++$
- Finally, the frequent itemsets (F_k) are identified for the pass:
$$F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{MIS}(c[1])\}$$

First pass over data

$L \leftarrow \text{init-pass}(M, T);$

- The algorithm first makes a pass over the data to record the support count of each item. It then follows the sorted order to find the first item i from the sorted items M that meets $\text{MIS}(i)$.
 - Item i will be inserted into the set of seeds L .
 - For each subsequent item j in M after i , if $j.\text{count}/n \geq \text{MIS}(i)$ then j is also inserted into L (as it already satisfies the minimum support requirements for a single item rule)
 $j.\text{count}$ is the support count of j ; n is the nr. of transactions in T .
- L is used by the function level2-candidate-gen

First pass over data: an example

- Consider the four items **1**, **2**, **3** and **4** in a data set. Their minimum item supports are:

$$MIS(1) = 10\% \quad MIS(2) = 20\%$$

$$MIS(3) = 5\% \quad MIS(4) = 6\%$$

- Assume our data set has **100** transactions. The first pass gives us the following support counts:

$$\{3\}.count = 6, \{4\}.count = 3,$$

$$\{1\}.count = 9, \{2\}.count = 25.$$

- Then $L = \{3, 1, 2\}$, and $F_1 = \{\{3\}, \{2\}\}$
- Item **4** is *not in* L because $4.count/n < MIS(3)$ ($= 5\%$),
- $\{1\}$ is *not in* F_1 because $1.count/n < MIS(1)$ ($= 10\%$).

The level2-candidate-gen function

Function level2-candidate-gen(L, ϕ)

// returns a superset of the set of all frequent 2-itemsets

1 $C_2 \leftarrow \emptyset$; // initialize the set of candidates

2 **for** each item l in L in the same order **do**

3 **if** $l.count/n \geq MIS(l)$ **then**

4 **for** each item h in L that is after l **do**

5 **if** $h.count/n \geq MIS(l)$ and $|sup(h) - sup(l)| \leq \phi$ **then**

 // we use $|sup(h) - sup(l)| \leq \phi$ because $sup(l)$ may not be lower

 // than $sup(h)$, although $MIS(l) \leq MIS(h)$.

6 $C_2 \leftarrow C_2 \cup \{\{l, h\}\}$; // insert the candidate $\{l, h\}$ into C_2

Note: we must use L rather than F_1 - because F_1 does not contain those items that may satisfy the MIS of an earlier item (in the sorted order) but not the MIS of itself (e.g., item 1 from previous example)

The MScandidate-gen function

	Function MScandidate-gen(F_{k-1}, φ)	// consists of a join and a pruning step
	1 $C_k \leftarrow \emptyset;$	// initialize the set of candidates
the join step	2 forall $f_1, f_2 \in F_k$	// find all pairs of frequent itemsets
	3 with $f_1 = \{i_1, \dots, i_{k-2}, i_{k-1}\}$	// that differ only in the last item
	4 and $f_2 = \{i_1, \dots, i_{k-2}, i'_{k-1}\}$	
	5 and $i_{k-1} < i'_{k-1}$ and $ sup(i_{k-1}) - sup(i'_{k-1}) \leq \varphi$ do	
	6 $c \leftarrow \{i_1, \dots, i_{k-1}, i'_{k-1}\};$	// join the two itemsets f_1 and f_2
	7 $C_k \leftarrow C_k \cup \{c\};$	// insert the candidate itemset c into C_k
the pruning step	8 for each $(k-1)$ -subset s of c do	// pruning of candidates
	9 if $(c[1] \in s)$ or $(MIS(c[2]) = MIS(c[1]))$ then	
	10 if $(s \notin F_{k-1})$ then	
	11 delete c from $C_k;$	// delete c from the set of candidates
	12 endifor	
	13 endfor	
	14 return $C_k;$	// return the generated candidates

The MScandidate-gen function

The function comprises two steps:

- **The join step** – like in the a-priori algorithm
 - **The pruning step:**
 - For each $(k-1)$ -subset s of c , if s is not in F_{k-1} , c can be deleted from C_k
 - Exception: s does not include $c[1]$ (there is only one such s), i.e., the first item of c , which has the lowest MIS value, is not in s .
- Even if s is not in F_{k-1} , we cannot delete c
(we cannot be sure that s does not satisfy $MIS(c[1])$, although we know that it does not satisfy $MIS(c[2])$, unless $MIS(c[2]) = MIS(c[1])$).

Specification of MIS values

Possibilities to specify the MIS values for the items:

- Assign a MIS value to each item according to its actual support / frequency in the data set T .
 - e.g., if the actual support of item i in T is $\mathbf{sup}(i)$, then the MIS value for i may be computed with $\lambda \times \mathbf{sup}(i)$, where λ is a parameter ($0 \leq \lambda \leq 1$) and is the same for all items in T .
- Group items into clusters (or blocks) of similar frequencies. Assign the same MIS value to all items from the same cluster.

Rule generation

- The following two lines in MS-Apriori algorithm are important for rule generation, but are not needed for the Apriori algorithm

if $c - \{c[1]\}$ is contained in t **then** $c.tailCount++$

- for the Apriori algorithm, if f is a frequent itemset and f_{sub} is a subset of f , then f_{sub} must also be a frequent itemset
- X In the case of MS-Apriori, it is not sufficient to record the support count of each frequent itemset in order to determine the confidence of the rules

Problems could occur only when the item that has the lowest MIS value in the itemset would be in the consequent of the rule

On multiple minsup rule mining

- Multiple minsup model **subsumes the single support model**.
- It is a **more realistic** model **for practical applications**.
- The model enables us to find **rare item rules without excessively producing meaningless rules** containing general frequent items.
- By setting the MIS values of the items to 100% (or more), we effectively **instruct the algorithms to not generate rules only involving these items**.

METHODS: ASSOCIATION RULES

Contents:

- *Market Basket Analysis*
- *Minimum Item Support*
- **Class Association Rules**
- *Sequential Pattern Mining*
- *Frequent Pattern Growth*

Contents:

- *Market Basket Analysis (MBA)*
 - *Association rules*
 - *Support, confidence, and lift*
 - *Algorithm APRIORI*
- *Minimum Item Support (MIS)*
 - *The MS-Apriori algorithm*
- **Class Association Rules (CAR)**
- *Sequential Pattern Mining*
 - *Basic concepts and an example*
 - *Generalized sequential pattern (GSP) mining*
- *Frequent Pattern Growth (FP-Growth)*
 - *FP-Tree construction*
 - *Frequent Itemset Generation*
 - *Prefix (path) sub-tree*
 - *Conditional FP-tree*

Mining class association rules (CAR)

- Usually, association rule mining does not involve any target (attributes).
- It finds all possible rules that exist in data, i.e., any item can appear as a condition or a consequent of a rule.
- However, in some applications, the user might be interested rather in the targets (attributes / classes).
 - E.g., the user has a set of text documents published on some known topics. He/she wants to find out what words are associated or correlated with each respective topic.

CAR - Problem definition

- Let T be a **transaction data set** consisting of n transactions.
- **Each transaction is also labeled with a class y .**
- Let I be the **set of all items** in T , Y be the **set of all class labels** and $I \cap Y = \emptyset$.
- A **class association rule (CAR)** is an implication of the form: $X \rightarrow y$, where $X \subseteq I$, and $y \in Y$.
- The definitions of **support** and **confidence** are the same like for usual association rules.

CAR – an example

A text document data set

doc 1: Student, Teach, School : Education
doc 2: Student, School : Education
doc 3: Teach, School, City, Game : Education
doc 4: Baseball, Basketball : Sport
doc 5: Basketball, Player, Spectator : Sport
doc 6: Baseball, Coach, Game, Team : Sport
doc 7: Basketball, Team, City, Game : Sport

Let ***minsup*** = 20% and ***minconf*** = 60%:

- The following are then two examples of class association rules:

Student, School $\rightarrow$ Education [sup= 2/7, conf = 2/2]

game $\rightarrow$ Sport [sup= 2/7, conf = 2/3]

CAR – the mining algorithm

- Unlike usual association rules, **CARs can be mined directly in one step.**
- The key operation is to **find all rule-items with a support above *minsup*.**
 - **Rule-items** have the form: ***(condset, y)***,
where **condset** is a set of items from I (i.e., $condset \subseteq I$),
and $y \in Y$ is the class label.
 - Each rule-item thus represents a **rule *condset* $\rightarrow y$** ,
 - The **a-priori** algorithm **can** be modified to **generate CARs**

CAR – multiple minimum class supports

- The idea of multiple minimum support can be used here, too.
- The user can specify **different minimum supports to different classes** (this effectively assigns a different minimum support to the rules for each respective class).
 - we can have, e.g., a data set with two classes, Yes and No. We may want:
 - rules of class Yes to have the minimum support of 5% and
 - rules of class No to have the minimum support of 10%.
- **By setting minimum class supports to 101% (or more), we prohibit generating rules for those classes.**
 - this is a useful trick in applications.

METHODS: ASSOCIATION RULES

Contents:

- *Market Basket Analysis*
- *Minimum Item Support*
- *Class Association Rules*
- **Sequential Pattern Mining**
- *Frequent Pattern Growth*

Contents:

- *Market Basket Analysis (MBA)*
 - *Association rules*
 - *Support, confidence, and lift*
 - *Algorithm APRIORI*
- *Minimum Item Support (MIS)*
 - *The MS-Apriori algorithm*
- *Class Association Rules (CAR)*
- **Sequential Pattern Mining**
 - **Basic concepts and an example**
 - **Generalized sequential pattern (GSP) mining**
- *Frequent Pattern Growth (FP-Growth)*
 - *FP-Tree construction*
 - *Frequent Itemset Generation*
 - *Prefix (path) sub-tree*
 - *Conditional FP-tree*

Sequential pattern mining

Association rule mining does not consider the order of transactions.

X In many applications, such orderings are significant!

- in market basket analysis, it is interesting to know whether people buy some items in sequence,
 - e.g., buying bed first and then bed sheets some time later.
- in Web usage mining, it is useful to find navigational patterns of users in a Web site based on the sequences of page visits of users

Basic concepts

Let $I = \{i_1, i_2, \dots, i_m\}$ be a set of items.

Sequence: An ordered list of itemsets.

Itemset/element: A non-empty set of items $X \subseteq I$. We denote a sequence s by $\langle a_1 a_2 \dots a_r \rangle$, where a_i is an itemset, which is also called an **element** of s .

- An element (or an itemset) of a sequence is denoted by $\{x_1, x_2, \dots, x_k\}$, where $x_j \in I$ is an item.
- We assume w.l.o.g. that the items in an element of a sequence are in **lexicographic order**.

Basic concepts (contd.)

Size: The **size** of a sequence is the number of elements (or itemsets) in the sequence.

Length: The **length** of a sequence is the number of items in the sequence.

- A sequence of length k is called **k -sequence**.

A sequence $s_1 = \langle a_1 a_2 \dots a_r \rangle$ is a **subsequence** of another sequence $s_2 = \langle b_1 b_2 \dots b_v \rangle$, or s_2 is a **supersequence** of s_1 , if there exist integers $1 \leq j_1 < j_2 < \dots < j_{r-1} < j_r \leq v$ such that $a_1 \subseteq b_{j_1}, a_2 \subseteq b_{j_2}, \dots, a_r \subseteq b_{j_r}$. We also say that s_2 **contains** s_1 .

An example

Let $I = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Sequence $\langle \{3\}\{4, 5\}\{8\} \rangle$ is **contained** in (or is a **subsequence** of) $\langle \{6\}\{3, 7\}\{9\}\{4, 5, 8\}\{3, 8\} \rangle$

- because $\{3\} \subseteq \{3, 7\}$, $\{4, 5\} \subseteq \{4, 5, 8\}$, and $\{8\} \subseteq \{3, 8\}$.
- However, $\langle \{3\}\{8\} \rangle$ is not contained in $\langle \{3, 8\} \rangle$ or vice versa.
- The size of the sequence $\langle \{3\}\{4, 5\}\{8\} \rangle$ is 3, and the length of the sequence is 4.

Objective

Given a set S of input data sequences (or sequence database), the **problem of mining sequential patterns** is to find all the sequences that have a **user-specified minimum support**.

Each such sequence is called a **frequent sequence**, or a **sequential pattern**.

The **support for a sequence** is the fraction of total data sequences in S that contains this sequence.

Example

Table 1. A set of transactions sorted by customer ID and transaction time

Customer ID	Transaction Time	Transaction (items bought)
1	July 20, 2005	30
1	July 25, 2005	90
2	July 9, 2005	10, 20
2	July 14, 2005	30
2	July 20, 2005	40, 60, 70
3	July 25, 2005	30, 50, 70
4	July 25, 2005	30
4	July 29, 2005	40, 70
4	August 2, 2005	90
5	July 12, 2005	90

Example (contd.)

Table 2. Data sequences produced from the transaction database in Table 1.

Customer ID	Data Sequence
1	$\langle\{30\} \{90\}\rangle$
2	$\langle\{10, 20\} \{30\} \{40, 60, 70\}\rangle$
3	$\langle\{30, 50, 70\}\rangle$
4	$\langle\{30\} \{40, 70\} \{90\}\rangle$
5	$\langle\{90\}\rangle$

Table 3. The final output sequential patterns

	Sequential Patterns with Support $\geq 25\%$
1-sequences	$\langle\{30\}\rangle, \langle\{40\}\rangle, \langle\{70\}\rangle, \langle\{90\}\rangle$
2-sequences	$\langle\{30\} \{40\}\rangle, \langle\{30\} \{70\}\rangle, \langle\{30\} \{90\}\rangle, \langle\{40, 70\}\rangle$
3-sequences	$\langle\{30\} \{40, 70\}\rangle$

Generalized Sequential Pattern algorithm (GSP)

GSP mining algorithm:

- generates sequential patterns
 - similar to the APRORI algorithm
-

Algorithm GSP(S)

```
1   $C_1 \leftarrow \text{init-pass}(S);$  // the first pass over  $S$ 
2   $F_1 \leftarrow \{\langle \{f\} \rangle \mid f \in C_1, f.\text{count}/n \geq \text{minsup}\};$  //  $n$  is the number of sequences in  $S$ 
3  for ( $k = 2; F_{k-1} \neq \emptyset; k++$ ) do // subsequent passes over  $S$ 
4     $C_k \leftarrow \text{candidate-gen-SPM}(F_{k-1});$ 
5    for each data sequence  $s \in S$  do // scan the data once
6      for each candidate  $c \in C_k$  do
7        if  $c$  is contained in  $s$  then
8           $c.\text{count}++;$  // increment the support count
9        end
10   end
11    $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{minsup}\}$ 
12 end
13 return  $\bigcup_k F_k;$ 
```

Candidate generation

Function candidate-gen-SPM(F_{k-1})

1. **Join step.** Candidate sequences are generated by joining F_{k-1} with F_{k-1} . A sequence s_1 joins with s_2 if the subsequence obtained by dropping the first item of s_1 is the same as the subsequence obtained by dropping the last item of s_2 . The candidate sequence generated by joining s_1 with s_2 is the sequence s_1 extended with the last item in s_2 . There are two cases:
 - the added item forms a separate element if it was a separate element in s_2 , and is appended at the end of s_1 in the merged sequence, and
 - the added item is part of the last element of s_1 in the merged sequence otherwise.

When joining F_1 with F_1 , we need to add the item in s_2 both as part of an itemset and as a separate element. That is, joining $\langle\{x\}\rangle$ with $\langle\{y\}\rangle$ gives us both $\langle\{x, y\}\rangle$ and $\langle\{x\} \{y\}\rangle$. Note that x and y in $\{x, y\}$ are ordered.

2. **Prune step.** A candidate sequence is pruned if any one of its $(k-1)$ -subsequence is infrequent (without minimum support).

An example

Table 4. Candidate generation: an example

Frequent 3-sequences	Candidate 4-sequences	
	after joining	after pruning
$\langle\{1, 2\} \{4\}\rangle$	$\langle\{1, 2\} \{4, 5\}\rangle$	$\langle\{1, 2\} \{4, 5\}\rangle$
$\langle\{1, 2\} \{5\}\rangle$	$\langle\{1, 2\} \{4\} \{6\}\rangle$	
$\langle\{1\} \{4, 5\}\rangle$		
$\langle\{1, 4\} \{6\}\rangle$		
$\langle\{2\} \{4, 5\}\rangle$		
$\langle\{2\} \{4\} \{6\}\rangle$		

METHODS: ASSOCIATION RULES

Contents:

- *Market Basket Analysis*
- *Minimum Item Support*
- *Class Association Rules*
- *Sequential Pattern Mining*
- **Frequent Pattern Growth**

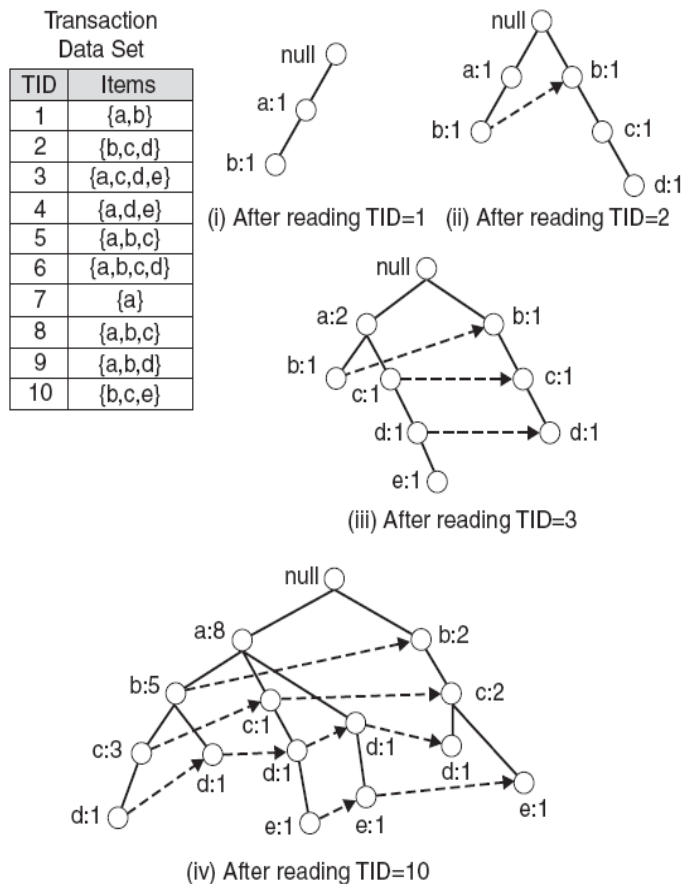
Contents:

- *Market Basket Analysis (MBA)*
 - *Association rules*
 - *Support, confidence, and lift*
 - *Algorithm APRIORI*
- *Minimum Item Support (MIS)*
 - *The MS-Apriori algorithm*
- *Class Association Rules (CAR)*
- *Sequential Pattern Mining*
 - *Basic concepts and an example*
 - *Generalized sequential pattern (GSP) mining*
- **Frequent Pattern Growth (FP-Growth)**
 - **FP-Tree construction**
 - **Frequent Itemset Generation**
 - **Prefix (path) sub-tree**
 - **Conditional FP-tree**

FP-Growth: Introduction

- **Apriori**: uses a generate-and-test approach
generates candidate itemsets and tests if they are frequent
 - Generation of candidate itemsets is expensive (both in space and time)
 - Support counting is expensive
 - Subset checking (computationally expensive)
 - Multiple Database scans (I/O)
- **FP-Growth**: allows frequent itemset discovery without candidate itemset generation. Two step approach:
 - **Step 1**: Build a compact data structure called the ***FP-tree***
 - Built using 2 passes over the data-set.
 - **Step 2**: Extracts frequent itemsets directly from the FP-tree
 - Traversal through FP-Tree

Core Data Structure: FP-Tree



- nodes correspond to items and have a counter
- FP-Growth reads 1 transaction at a time and maps it to a path
- fixed order is used, so **paths can overlap when transactions share items** (when they have the same prefix).
- in this case, counters are incremented
- pointers are maintained between nodes containing the same item, creating singly linked lists (dotted lines)
- the more paths that overlap, the higher the compression.** FP-tree may fit in memory.
- frequent itemsets extracted from the FP-Tree.

Step 1: FP-Tree Construction (Example)

FP-Tree is constructed using 2 passes over the data-set:

- **Pass 1:**

- Scan data and find support for each item.
- Discard infrequent items.
- Sort frequent items in decreasing order based on their support.
 - For our example: ***a, b, c, d, e***
 - Use this order when building the FP-Tree, so common prefixes can be shared.

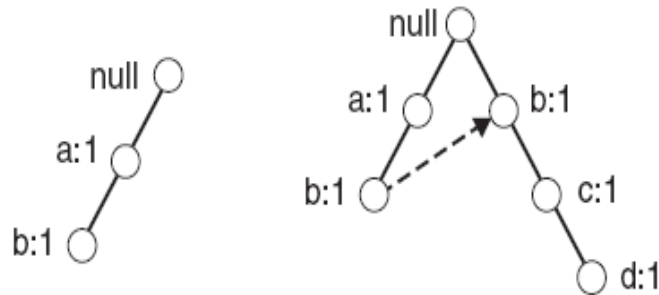
Step 1: FP-Tree Construction (Example)

- **Pass 2: construct the FP-Tree** (see the diagram on the next slide)
 - Read transaction 1: { **a** , **b** }
 - Create 2 nodes **a** and **b** and the path **null -> a -> b**.
 - Set counts of **a** and **b** to 1.
 - Read transaction 2: { **b** , **c** , **d** }
 - Create 3 nodes for **b**, **c** and **d** and the path **null -> b -> c -> d**.
 - Set counts to 1.
 - Note that although transaction 1 and 2 share **b**, the paths are disjoint as they don't share a common prefix. Add the link between the **b**'s.
 - Read transaction 3: { **a** , **c** , **d** , **e** }
 - It shares common prefix item **a** with transaction 1 so the path for transaction 1 and 3 will overlap and the frequency count for node **a** will be incremented by 1. Add links between the **c**'s and **d**'s.
 - Continue until all transactions are mapped to a path in the FP-tree.

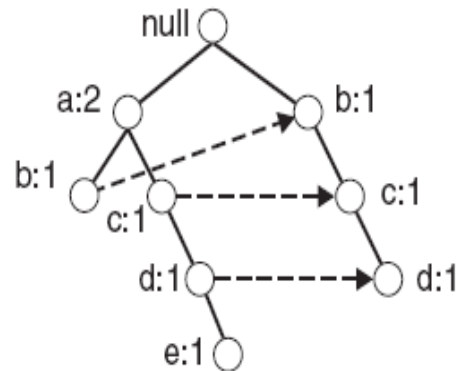
Step 1: FP-Tree Construction (Example)

Transaction
Data Set

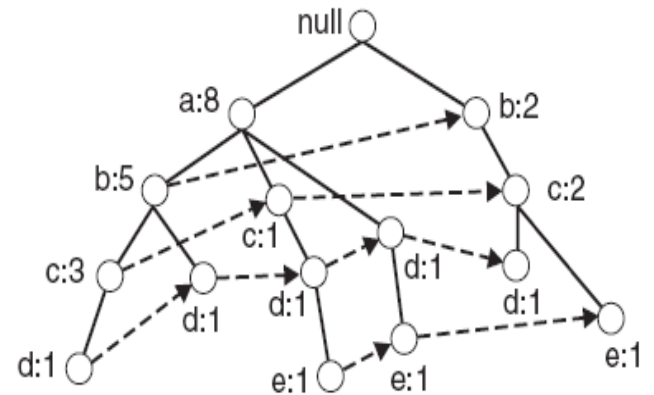
TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}



(i) After reading TID=1 (ii) After reading TID=2



(iii) After reading TID=3



(iv) After reading TID=10

FP-Tree size

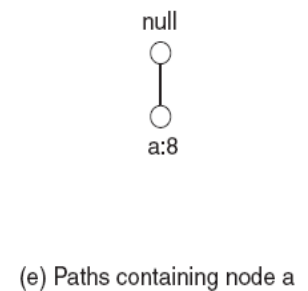
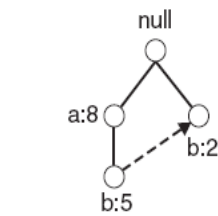
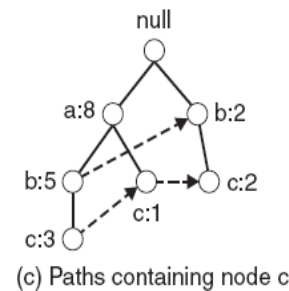
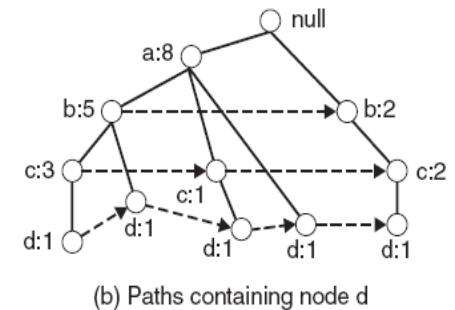
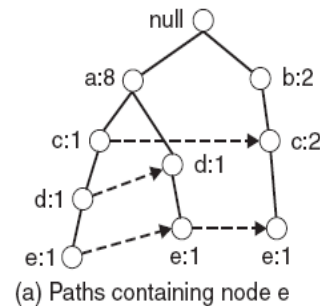
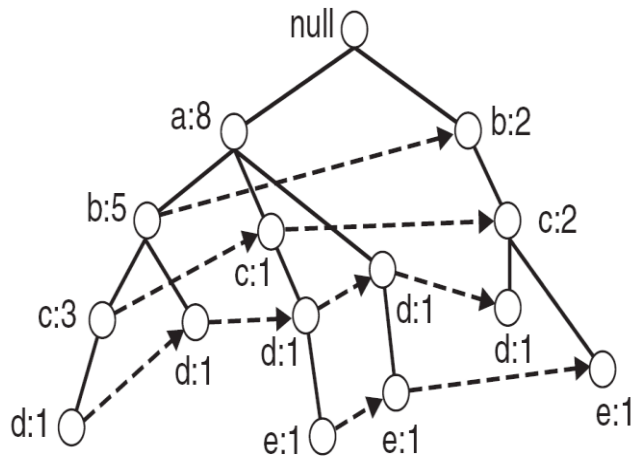
- The FP-Tree usually has a smaller size than the uncompressed data - typically many transactions share items (and hence prefixes).
 - **Best case scenario:** all transactions contain the same set of items
 - 1 path in the FP-tree
 - **Worst case scenario:** every transaction has a unique set of items (no items in common)
 - the size of the FP-tree is at least as large as the original data.
 - the storage requirements for the FP-tree are higher (need to store the pointers between the nodes and the counters)
 - The size of the FP-tree depends on how the items are ordered
 - ordering by decreasing support is typically used but it does not always lead to the smallest tree (it's a heuristic).

Step 2: Frequent Itemset Generation

- FP-Growth extracts frequent itemsets from the FP-tree.
- Bottom-up algorithm from the leaves towards the root
 - Divide and conquer: first look for frequent itemsets ending in **e**, then **de**, etc. . . then **d**, then **cd**, etc. . .
- First, extract prefix path sub-trees ending in an item. (hint: use the linked lists)

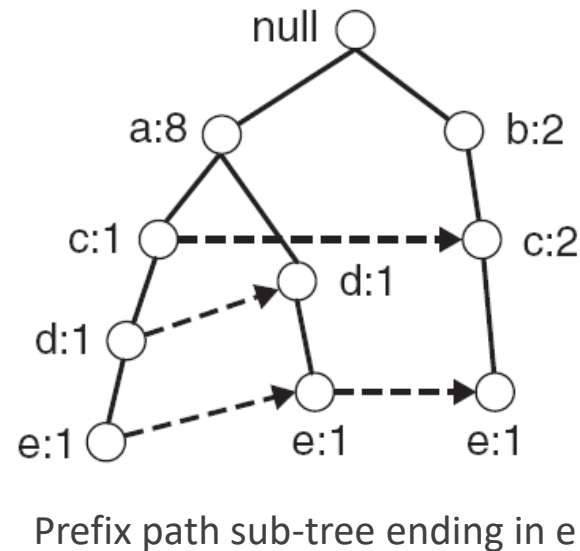
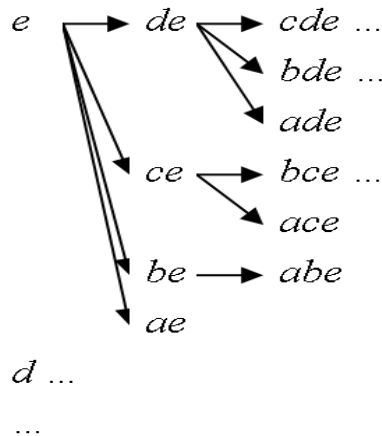
Step 2: Frequent Itemset Generation

- Complete FP-tree
- Example: prefix path sub-trees



Step 2: Frequent Itemset Generation

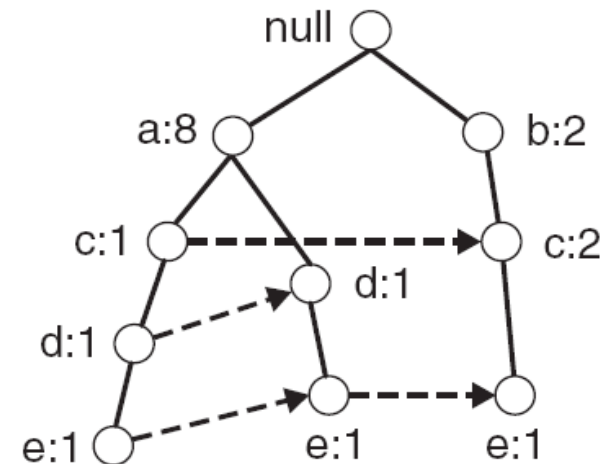
- Each prefix path sub-tree is processed recursively to extract the frequent itemsets. Solutions are then merged.
 - e.g., the **prefix path sub-tree** for **e** will be used to extract frequent item-sets ending in **e**, then in **de**, **ce**, **be** and **ae**, then in **cde**, **bde**, **cde**, etc.
 - divide and conquer approach



Example

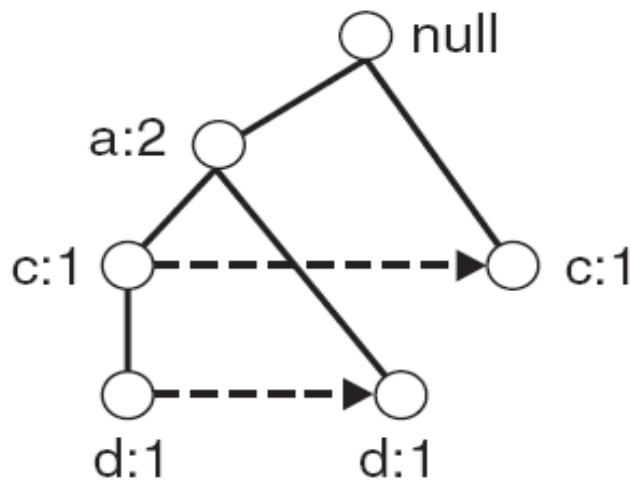
Let ***minSup* = 2** and extract all frequent itemsets containing ***e***.

1. Obtain the prefix path sub-tree for ***e***:
2. Check if ***e*** is a frequent item by adding the counts along the linked list (dotted line). If so, extract it.
 - Yes, ***count* = 3** so ***{e}*** is extracted as a frequent itemset.
3. As ***e*** is frequent, find frequent itemsets ending in ***e***, i.e., ***de***, ***ce***, ***be*** and ***ae***.
 - ~ decompose the problem recursively.
 - to do this, we must **first obtain the conditional FP-tree for *e***.



Conditional FP-Tree

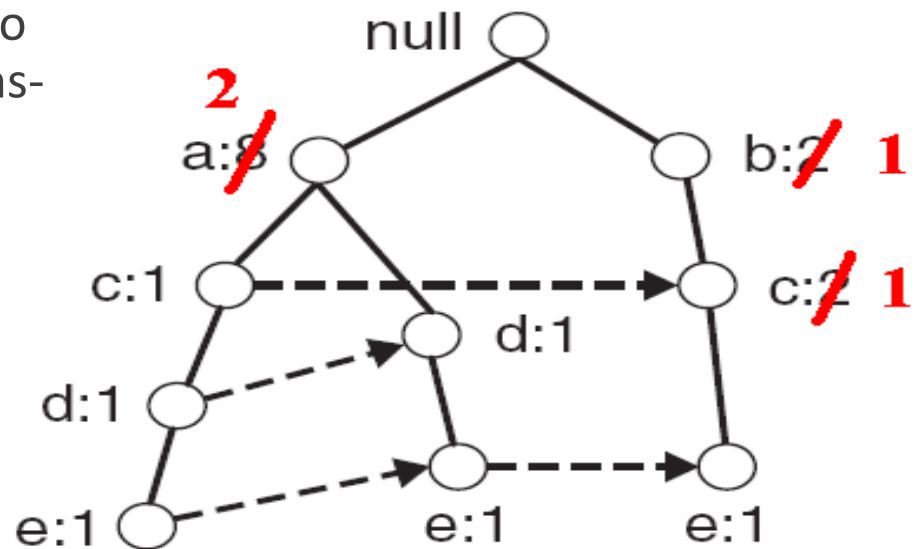
- The FP-Tree that would be built if we only consider transactions containing a particular itemset (and then removing that itemset from all transactions).
- **Example:** FP-Tree conditional on **e**.



TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d, e }
4	{a,d, e }
5	{a,b,e}
6	{a,b,c,d}
7	{a}
8	{a,b,e}
9	{a,b,d}
10	{b,c, e }

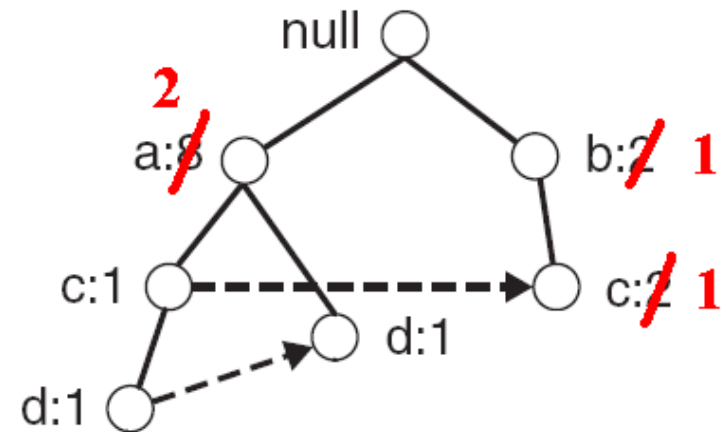
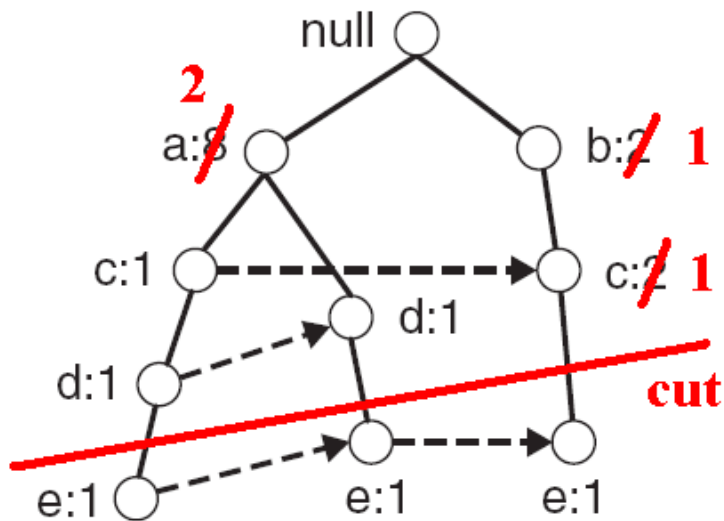
Conditional FP-Tree

- To obtain the *conditional FP-tree* for **e** from the *prefix sub-tree* ending in **e**:
 - Update the support counts along the prefix paths (from **e**) to reflect the number of transactions containing **e**.
 - **b** and **c** should be set to 1 and **a** to 2.



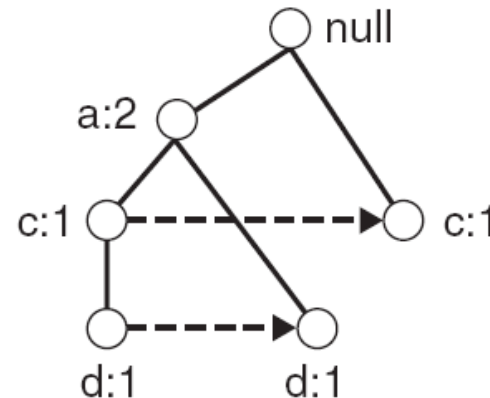
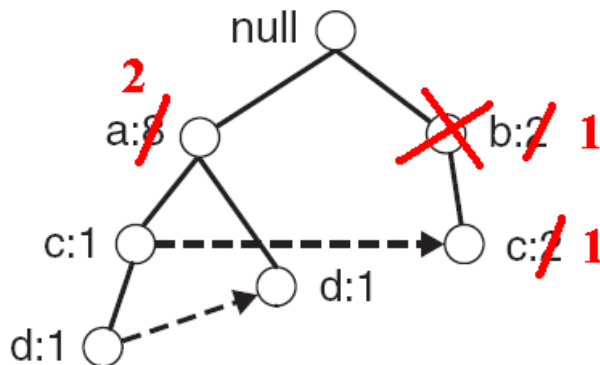
Conditional FP-Tree

- To obtain the **conditional FP-tree** for ***e*** from the **prefix sub-tree** ending in ***e***:
 - Remove the nodes containing ***e*** - information about node ***e*** is no longer needed because of the previous step



Conditional FP-Tree

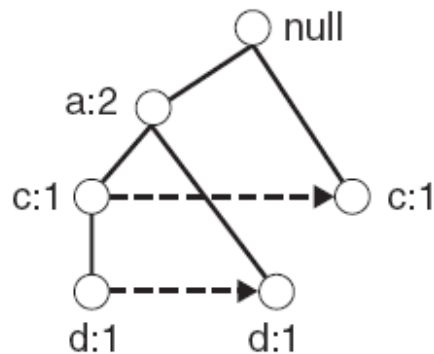
- To obtain the *conditional FP-tree* for **e** from the *prefix sub-tree* ending in **e**:
 - remove infrequent items (nodes) from the prefix paths
 - e.g., **b** has a support of 1 (note this really means **be** has a support of 1). i.e., there is only 1 transaction containing **b** and **e** so **be** is infrequent - can remove **b**.



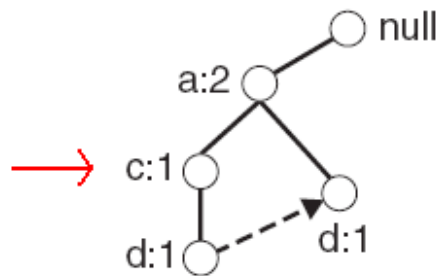
- Question: Why were **c** and **d** not removed?

Example (continued)

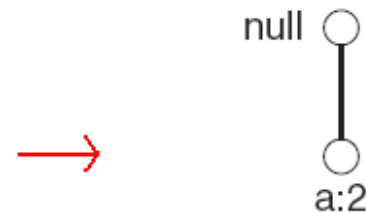
- Use the conditional FP-tree for **e** to find frequent itemsets ending in **de**, **ce** and **ae**
 - Note that **be** is not considered as **b** is not in the conditional FP-tree for **e**.
 - For each of them (e.g., **de**), find the prefix paths from the conditional tree for **e**, extract frequent itemsets, generate conditional FP-tree, etc... (recursive)
 - Example:** $e \rightarrow de \rightarrow ade$ ($\{d, e\}$, $\{a, d, e\}$ are found to be frequent)



Conditional FP-tree for e



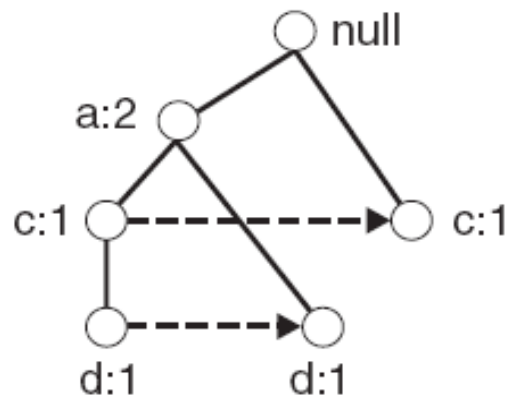
Prefix paths ending in de



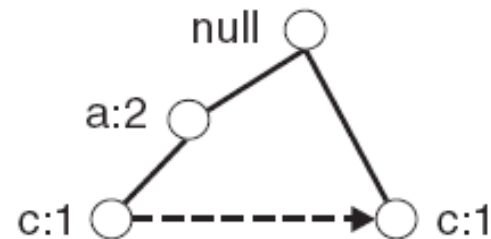
Conditional FP-tree for de

Example (continued)

- Use the conditional FP-tree for **e** to find frequent itemsets ending in **de**, **ce** and **ae**
 - Example: $e \rightarrow ce$ ($\{c, e\}$ is found to be frequent)



Conditional FP-tree for e



Prefix paths ending in ce

- etc. ... (**ae**, then do the whole thing for **b**,..., etc.)

Example: Result

- Frequent itemsets found (ordered by suffix and order in which they were found):

Suffix	Frequent Itemsets
e	{e}, {d,e}, {a,d,e}, {c,e}, {a,e}
d	{d}, {c,d}, {b,c,d}, {a,c,d}, {b,d}, {a,b,d}, {a,d}
c	{c}, {b,c}, {a,b,c}, {a,c}
b	{b}, {a,b}
a	{a}

FP-Growth Algorithm: Discussion

■ Advantages of FP-Growth:

- only 2 passes over the data-set
- „compresses“ the data-set
- no candidate generation like for Apriori and much faster

■ Disadvantages of FP-Growth

- FP-Tree may not fit in memory!!
- FP-Tree is expensive to build
 - Takes time to build, but afterwards, frequent itemsets are read off easily.
 - Time is wasted, as the only pruning that can be done on **single items**.
 - Support can be calculated only after the entire data is added to the FP-Tree.